The Nature of Code, but Python



Introduction



https://tisch.nyu.edu/itp/news/fall-2024/daniel-shiffman--itp-faculty-member--launches-new-book---the-nat

Think:

Python wearing a Processing costume,
and nobody at the Nature of Code party notices.

Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch00_randomness/00.06/perlin_noise_walker.py...

File Edit View Run Tools Help py5

perlin_noise_walker.py

```
        self.tx = 0
        self.ty = 10_000

    def step(self) -> None:
        # x- and y-position mapped from noise.
        self.x = remap(noise(self.tx), 0, 1, 0, width)
        self.y = remap(noise(self.ty), 0, 1, 0, height)
        # Move forward through time.
        self.tx += 0.01
        self.ty += 0.01

    def show(self) -> None:
        stroke_weight(2)
        fill(127)
        stroke(0)
        circle(self.x, self.y, 48)

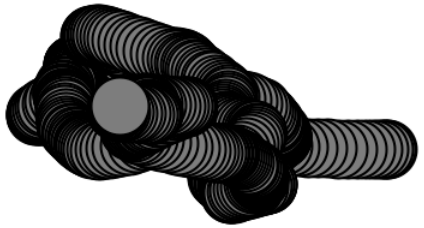

def setup():
    global walker
    size(640, 240)
    walker = Walker()
    background(255)


def draw():
```

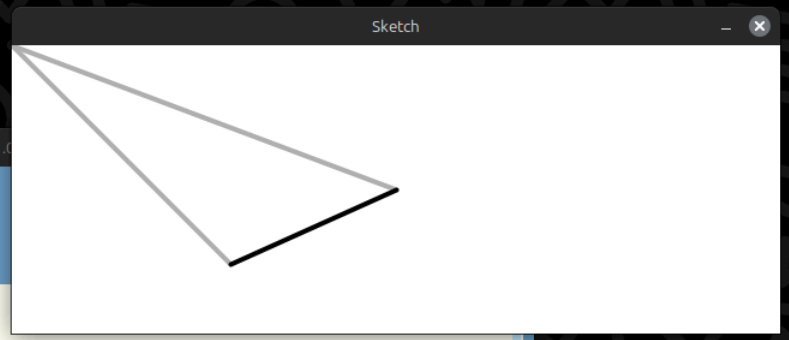
Shell

Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

Sketch



00.06 (perlin_noise_walker.py)



Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch01_vectors/01.0

File Edit View Run Tools Help py5

vector_subtraction.py

```
# https://natureofcode.com/vectors/#example-13-vector-subtraction


def setup():
    size(640, 240)


def draw():
    background(255)

    # Two vectors, one for mouse location and one for the center of the window.
    mouse = Py5Vector2D(mouse_x, mouse_y)
    center = Py5Vector2D(width / 2, height / 2)

    # Draw the original two vectors.
    stroke(175)
    stroke_weight(4)
    line(0, 0, mouse.x, mouse.y)
    line(0, 0, center.x, center.y)
```

Shell

Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

01.04 (vector_multiplication.py)

Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch02_forces/02.07/attraction_many_movers.py @ 55 : 1

File Edit View Run Tools Help py5

attraction_many_movers.py

```
# https://natureofcode.com/forces/#example-27-attraction-with-many-movers

from mover import Mover
from attractor import Attractor

G = 1.0  # A gravitational constant (for global scaling)


def setup():
    global movers, attractor
    size(640, 240)

    # Now there are 10 movers!
    movers = [
      # Each mover is initialized randomly.
      Mover(random(width), random(height), random(0.5, 3))
      for _ in range(10)
    ]

    # JS version Mover class hardcodes this.velocity
    for m in movers:
        m.velocity = Py5Vector2D(1, 0)

    attractor = Attractor()  # Initialize the Attractor object.


def draw():
    background(255)

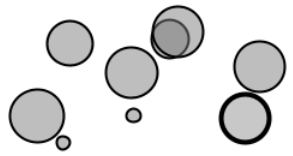
    attractor.show()  # Draw the Attractor object.

    # Apply the attraction force from the attractor on the mover(s).
    for mover in movers:
        # Calculate an attraction force for each Mover object.
        force = attractor.attract(mover, G)
```

Shell

Local Python 3 • /home/me/Desktop/thonny/env/b

Sketch



02.07 (attraction_many_movers.py)

Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch03_oscillation/03.09/the_wave.py @ 30 : 1

File Edit View Run Tools Help py5

the_wave.py

```
# https://natureofcode.com/oscillation/#example-39-the-wave

DELTA_ANGLE = 0.2
start_angle = 0


def setup():
    size(640, 240)


def draw():
    global start_angle
    background(255)

    # Each time through draw(), the angle that increments is set to start_angle.
    angle = start_angle

    for x in range(0, width + 1, 24):

        y = remap(sin(angle), -1, 1, 0, height)

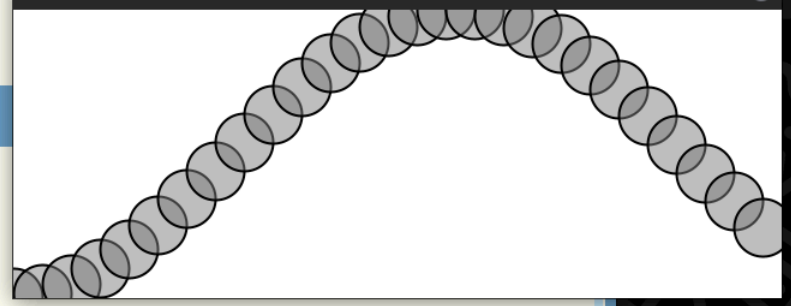
        stroke(0)
        stroke_weight(2)
        fill(127, 127)
        circle(x, y, 48)

        anale += DELTA ANGLE
```

Shell

Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

Sketch



03.09 (the_wave.py)

Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch04_particle_systems/04.09/additive_smoke_system.py @ 43 : 1

File Edit View Run Tools Help py5

additive_smoke_system.py

```
# https://natureofcode.com/particles/#example-49-additive-blending

from emitter import Emitter


def setup():
    global emitter, img
    size(640, 240, P3D)  # Enable the OpenGL-compatible 3D renderer.
    img = load_image('texture.png')  # Load the PNG.
    emitter = Emitter(width / 2, height - 75, img)

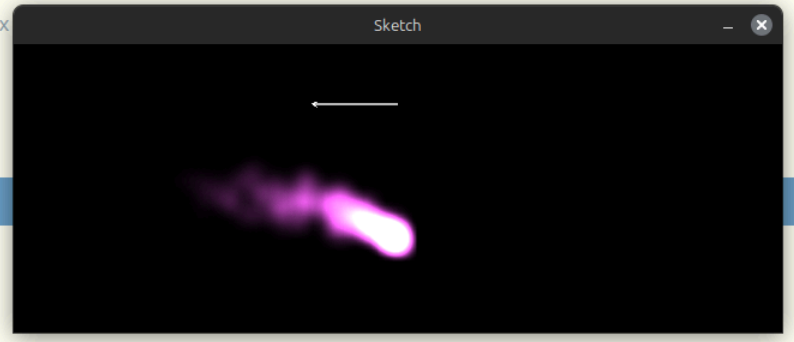

def draw():
    # Use additive blending.
    blend_mode(ADD)
    # Additive glow needs a dark background (won't work over white/bright).
    background(0)

    # The wind force direction is based on mouse_x
    dx = remap(mouse_x, 0, width, -0.2, 0.2)
    wind = Py5Vector2D(dx, 0)

    emitter.apply_force(wind)
    emitter.run()
```

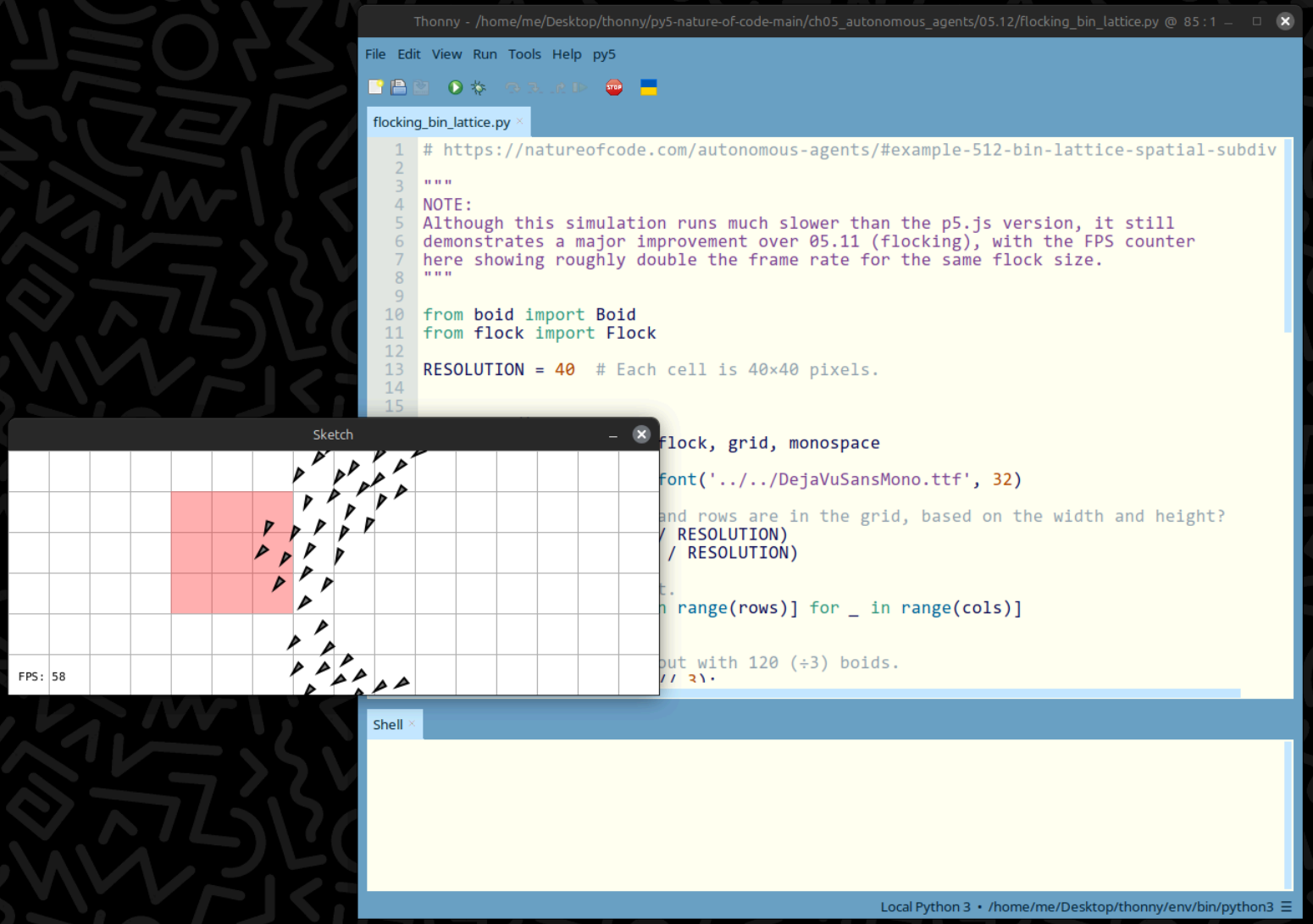
Shell

Sketch

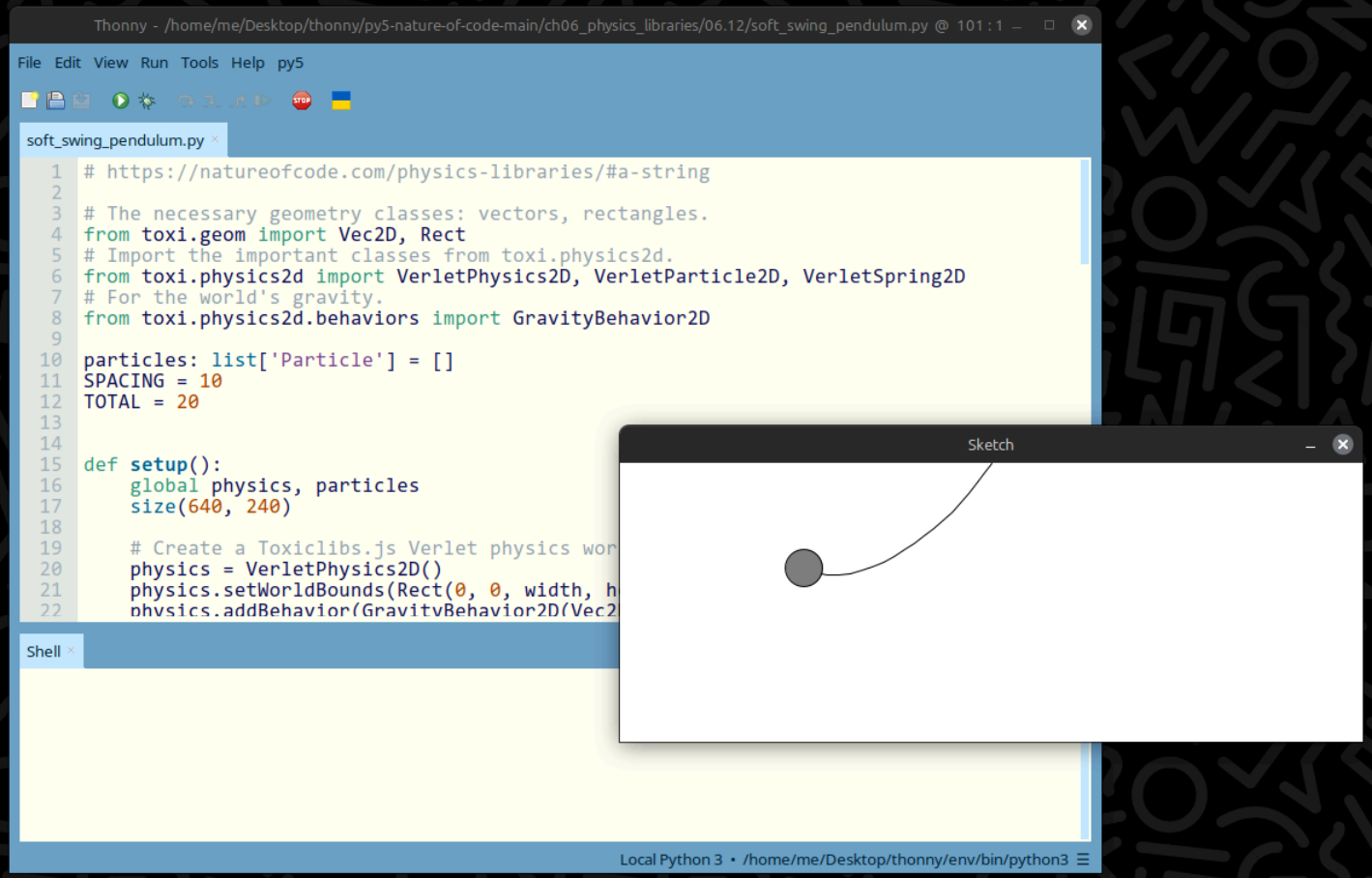


Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

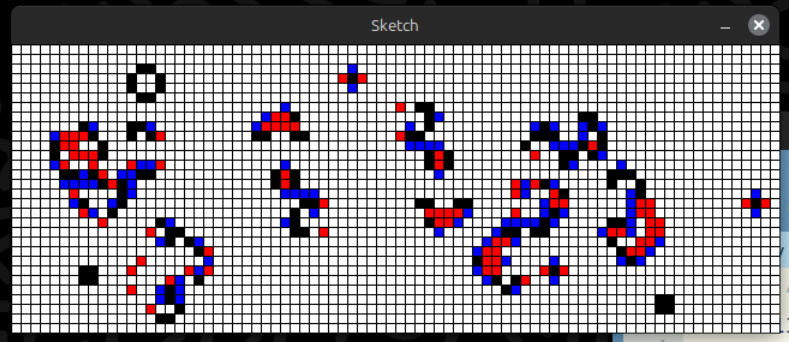
04.09 (additive_smoke_system.py)



05.12 (flocking_bin_lattice.py)



06.12 (soft_swing_pendulum.py)



Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch07_cellular_automata/07.03/oop_game_of_life.py @ 62 : 1

Tools Help py5

```
//natureofcode.com/cellular-automata/#object-oriented-cells

l import Cell

W = 8


def setup():
    global board, columns, rows
    size(640, 240)

    columns = width // W
    rows = height // W

    board = create_2d_array(columns, rows)

    for i in range(1, columns - 1):
        for j in range(1, rows - 1):
            # Start each cell with a 0 or 1.
            board[i][j] = Cell(random_int(), i * W, j * W, W)
            # With no arguments, random_int() return zero or one.


def draw():
    # Loop but skip the edge cells.
    for i in range(1, columns - 1):
```

Shell

Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

07.03 (oop_game_of_life.py)

Thonny - /home/me/Desktop/thonny/py5-nature-of-code-main/ch08_fractals/08.06/recursive_tree.py @ ...

File Edit View Run Tools Help py5

recursive_tree.py

```python
# https://natureofcode.com/fractals/#the-deterministic-version


def setup():
    size(640, 240)


def draw():
    global angle
    background(255)

    # Map the angle to range from 0° to 90° (HALF_PI) according to
    angle = remap(mouse_x, 0, width, 0, HALF_PI)

    # Start the tree from the bottom of the canvas.
    translate(width / 2, height)
    stroke(0)
    stroke_weight(2)
    branch(80)


def branch(length: float) -> None:  # Each branch receives its len
    line(0, 0, 0, -length)  # Draw the branch.
    translate(0, -length)   # Translate to the end.
    length *= 0.67          # Each branch's length shrinks by one-

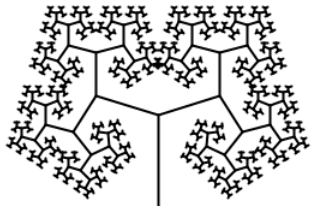
    if length > 2:  # Exit condition for the recursion!
        push()
        # Rotate to the right and branch again.
        rotate(angle)
        branch(length)  # Subsequent calls to branch() include a l
        pop()

        push()
        # Rotate to the left and branch again.
```

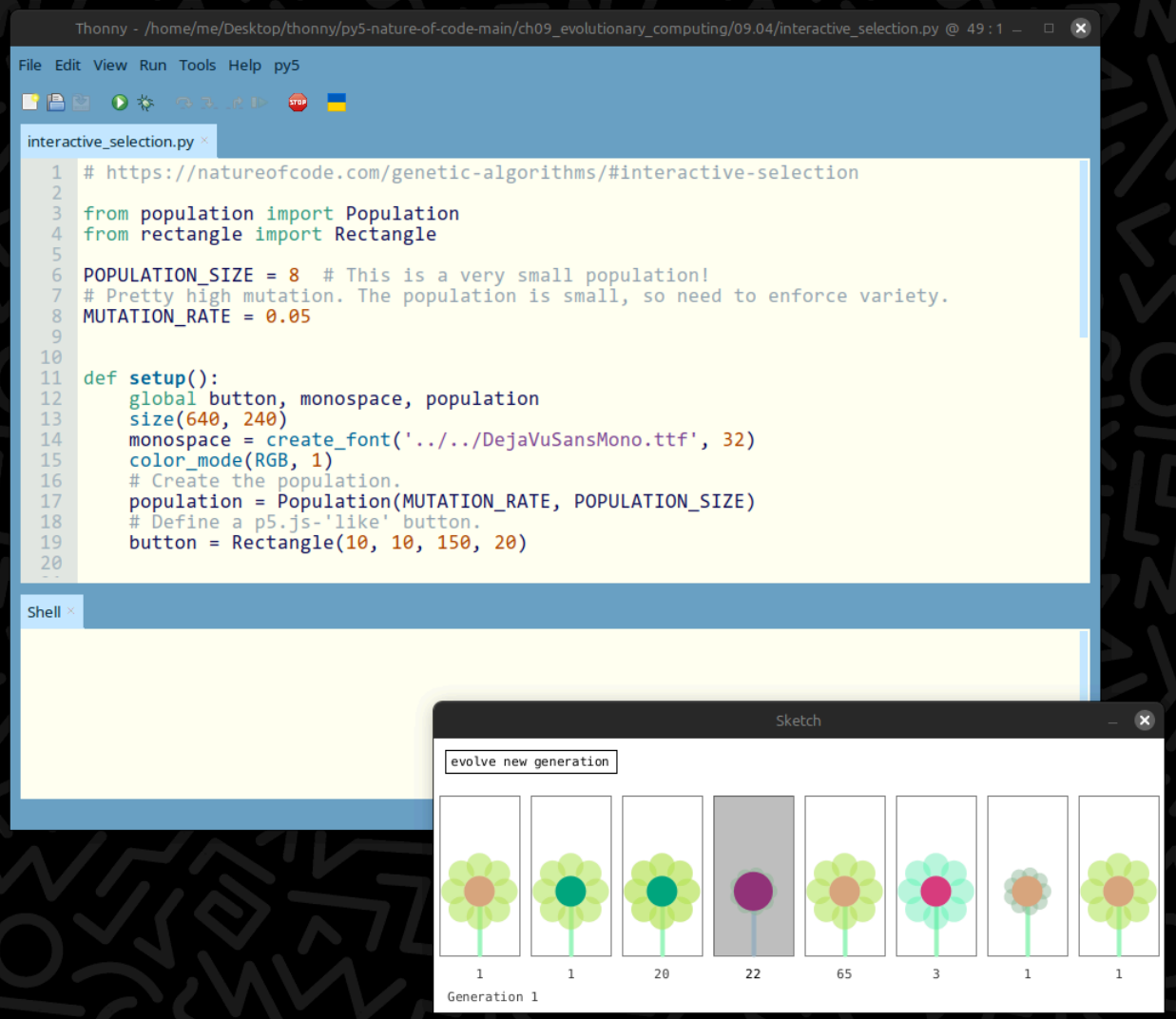
Shell

Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

Sketch



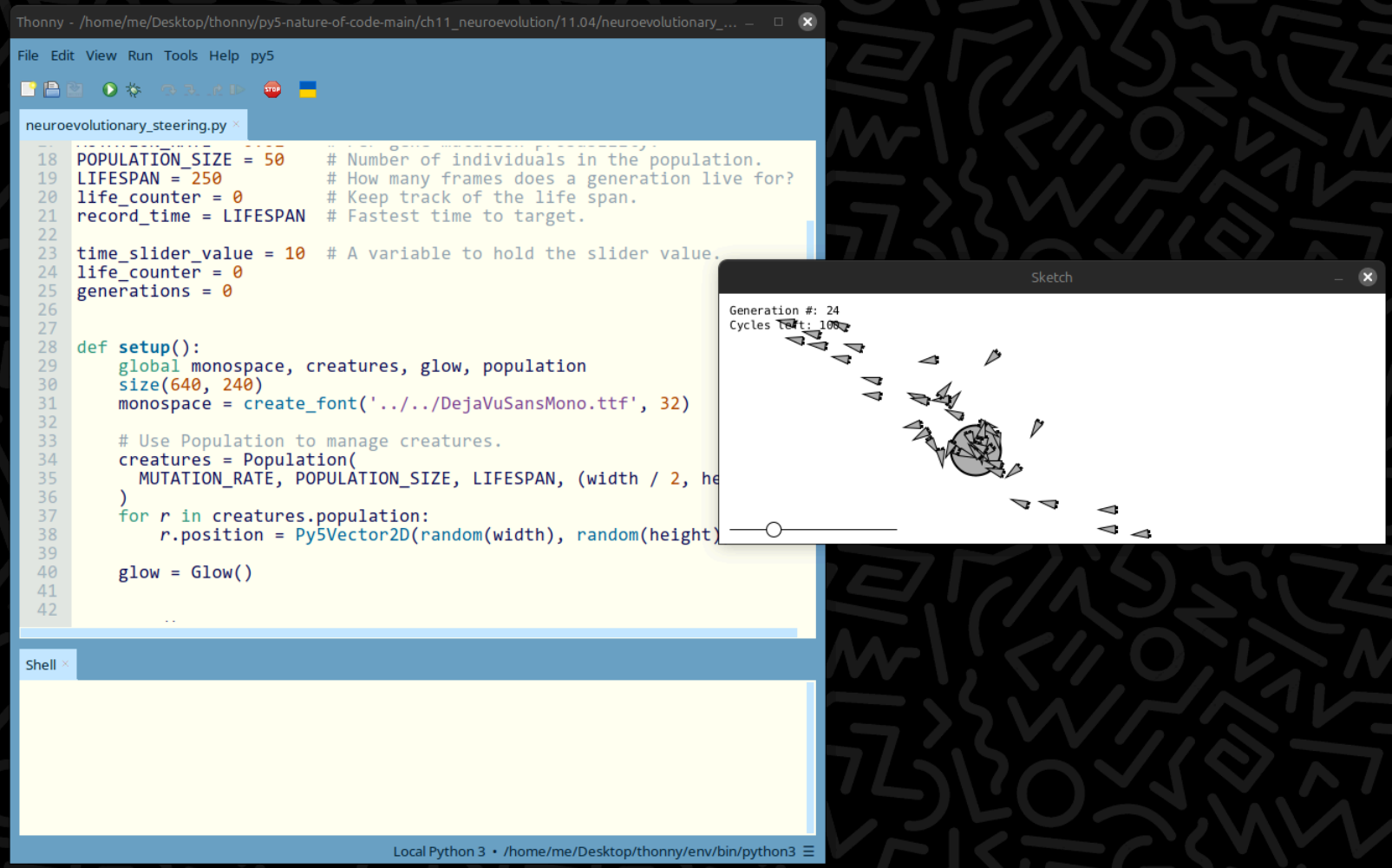
08.06 (recursive_tree.py)



09.04 (interactive_selection.py)



10.02 (gesture_classifier.py)

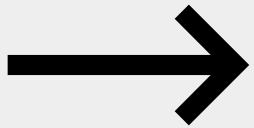


11.04 (neuroevolutionary_steering.py)

Introduction » Setup

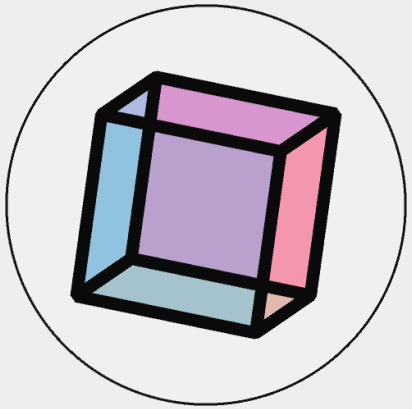
JS to Python, and IDE configuration ...


JS

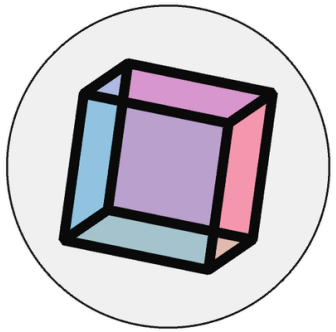




p5.js →



https://py5coding.org



Search Ctrl + K

- Welcome to py5!

About py5

- What is py5?
- py5 Community
- How To Contribute
- py5 maintainer's blog

Getting Started

- Install py5
- Tips for Processing Java Users
- Special Notes for macOS Users
- The Five py5 Modes

Welcome to py5!

py5 is a new version of Processing for Python. The goal of py5 is to create a version of Processing that is integrated into the Python ecosystem. Built into the library are thoughtful choices about how to best get py5 to work with other popular Python libraries and tools such as Jupyter, numpy, shapely, trimesh, matplotlib, and Pillow.

py5 is an excellent choice for educators looking to teach Python in the context of creative coding and is currently used in classrooms all around the world. This website's documentation includes introductory tutorials as well as extensive reference documentation, complete with example code.

Basic Example

Here is a basic example of a py5 Sketch:

```
def setup():
    size(400, 400)
    rect_mode(CENTER)


def draw():
    square(mouse_x, mouse_y, 10)


def mouse_clicked():
    fill(random_int(255), random_int(255), random_int(255))


run_sketch()
```

https://py5coding.org

Introduction » py5 Features

- Modern Python: full Python 3 support, including C-extension libraries, including native integration with NumPy, Pillow, Shapely, etc.
- Processing engine: Java Processing graphics backend via JPype, and can load and use many existing Java Processing libraries
- Any editor: works in Jupyter Notebooks, VS Code, etc. with flexible modes: static & imported (beginner), module, or class
- Fun features to explore: pixel manipulation using NumPy arrays, live coding, Matplotlib, Shapely geometries, Pandas/Polars etc.

Introduction » Run py5

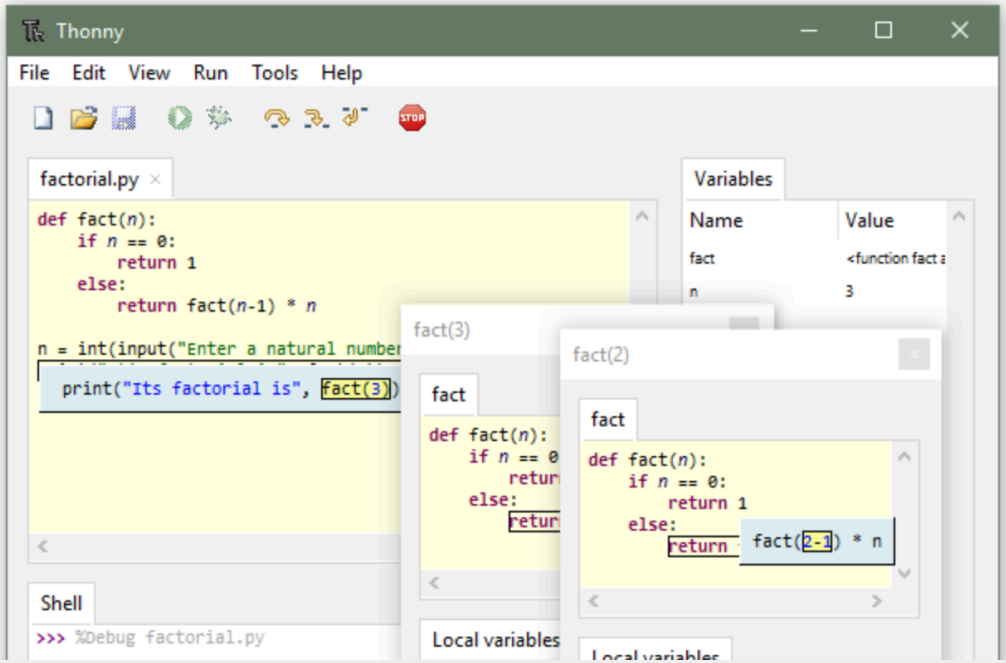
- Standard python script: VS Code, PyCharm, terminal, etc.
- Jupyter notebook: via standard ipykernel, or py5 kernel
- Thonny-py5mode: Thonny IDE + py5 plugin ←

Thonny 5 is dedicated to Ukraine fighting the Russian invasion.
🇺🇦 Please support Ukraine! 🇺🇦

Thonny

Python IDE for beginners

Download version 5.0.0 for
Windows • Mac • Linux



https://thonny.org

Introduction » Thonny Features

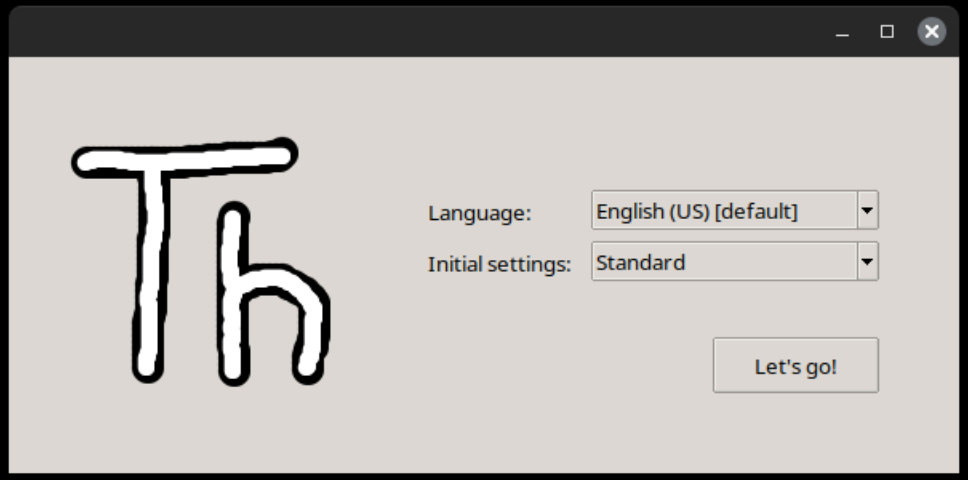
- Beginner-focused: minimal, uncluttered interface designed for learning
- Bundled Python: includes a built-in Python 3 runtime
- Cross-platform: runs on Linux, macOS, Windows
- Fully portable: runs directly from a USB drive without admin rights
- Several neat panels: interactive shell, plotter, outline, and more

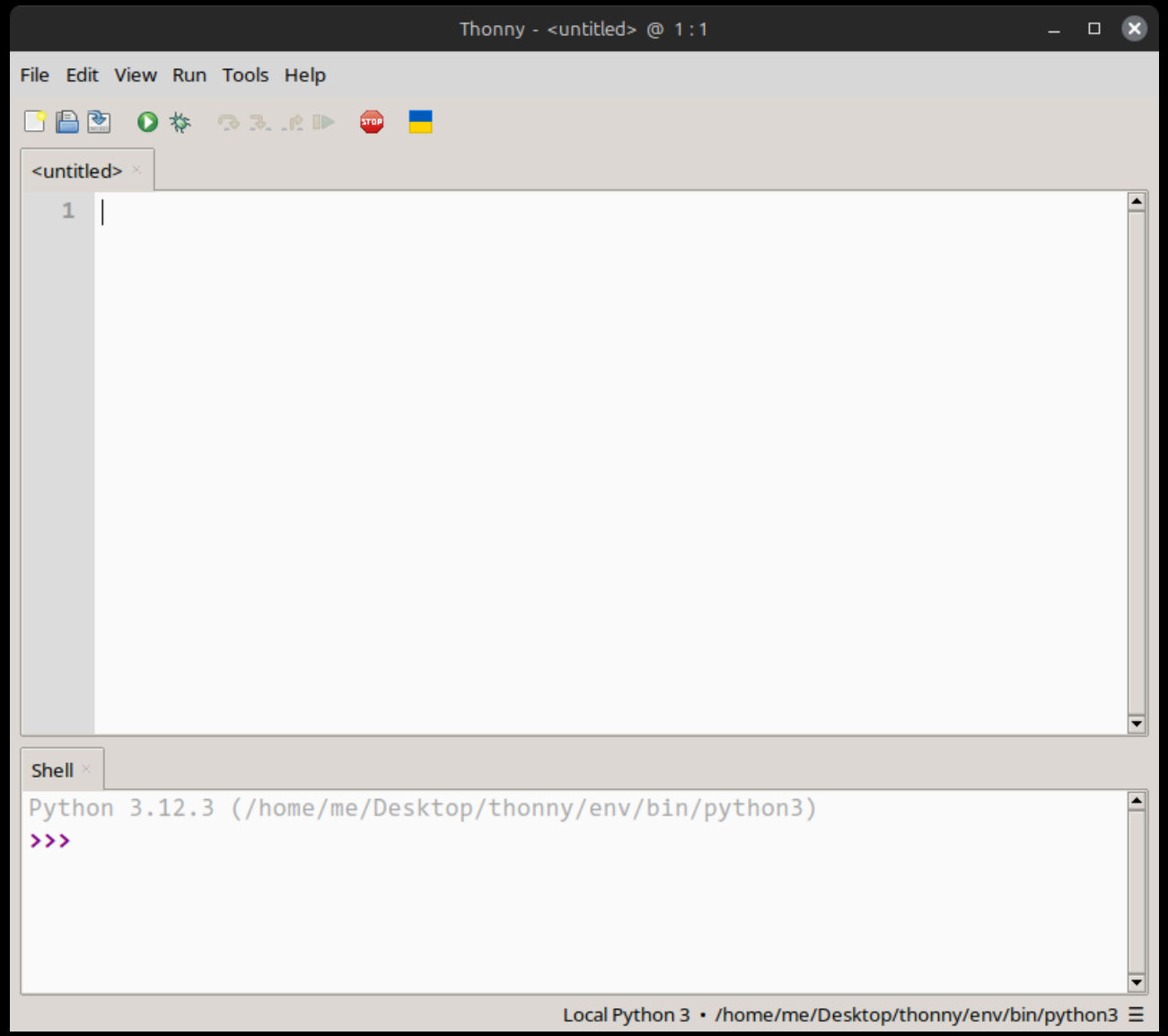
Introduction » Thonny-py5mode Installation

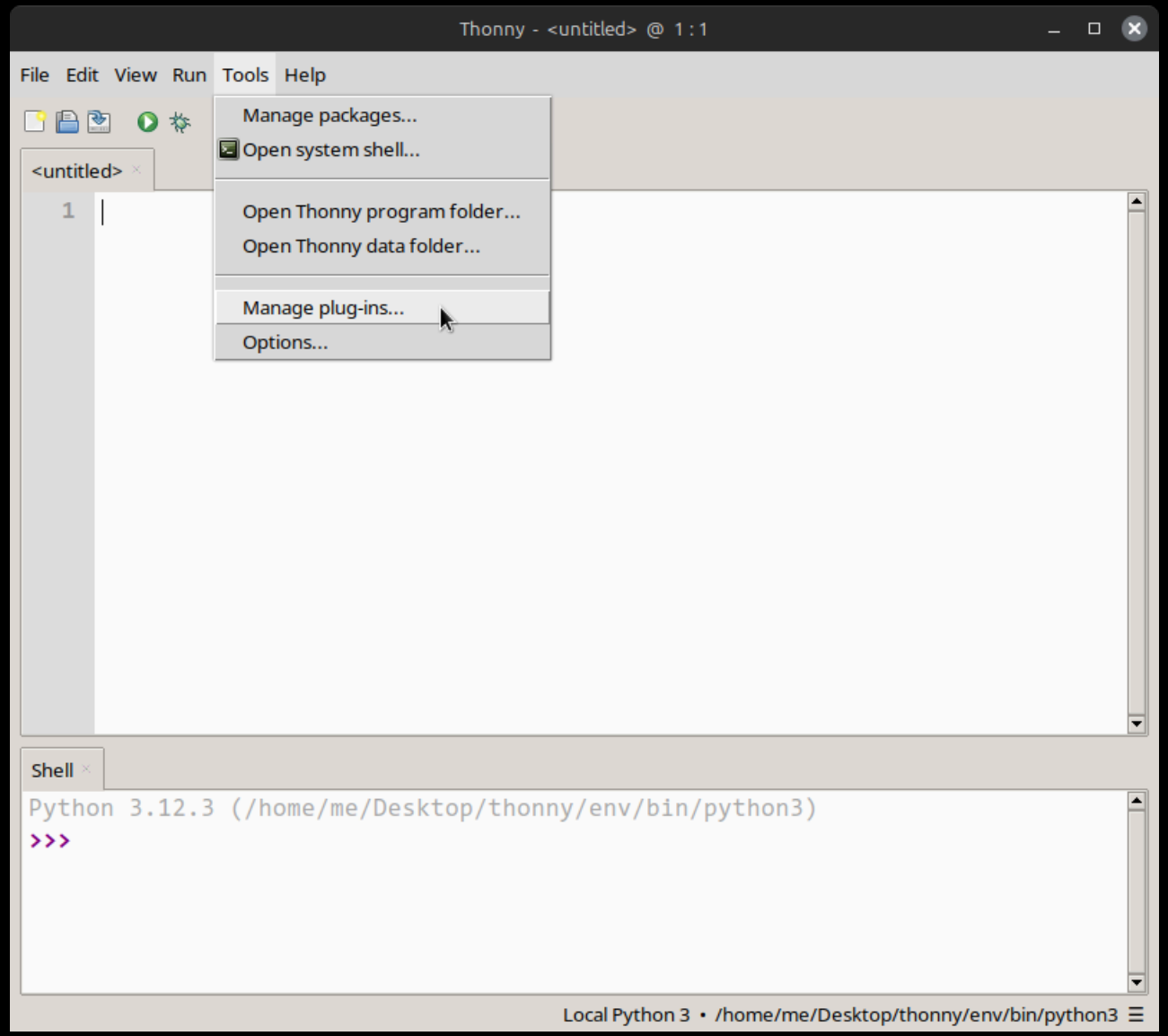
A plugin that turns the Thonny IDE into a Python 3 ~~shameless ripoff~~ replacement for the original Processing IDE.

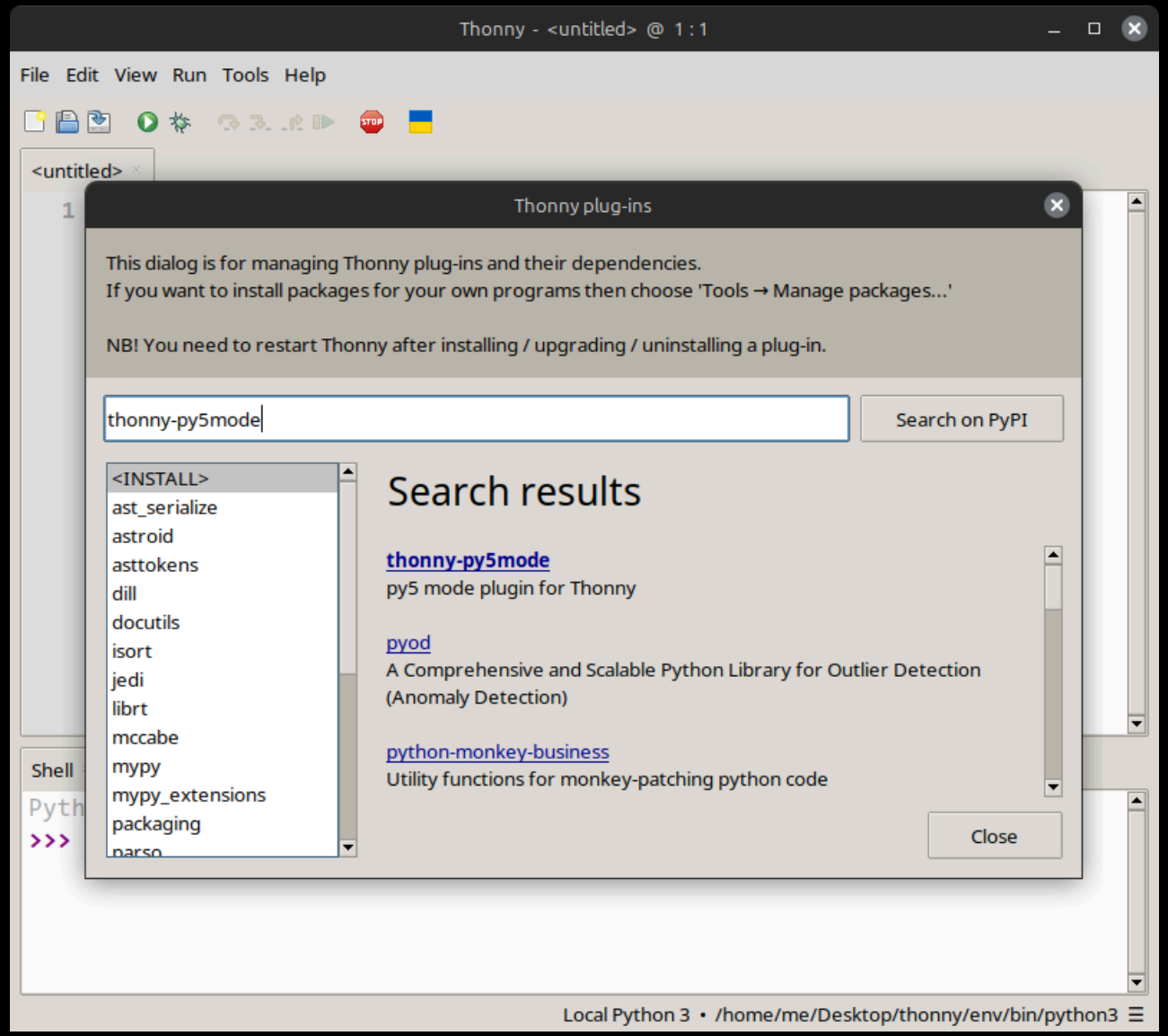
Process

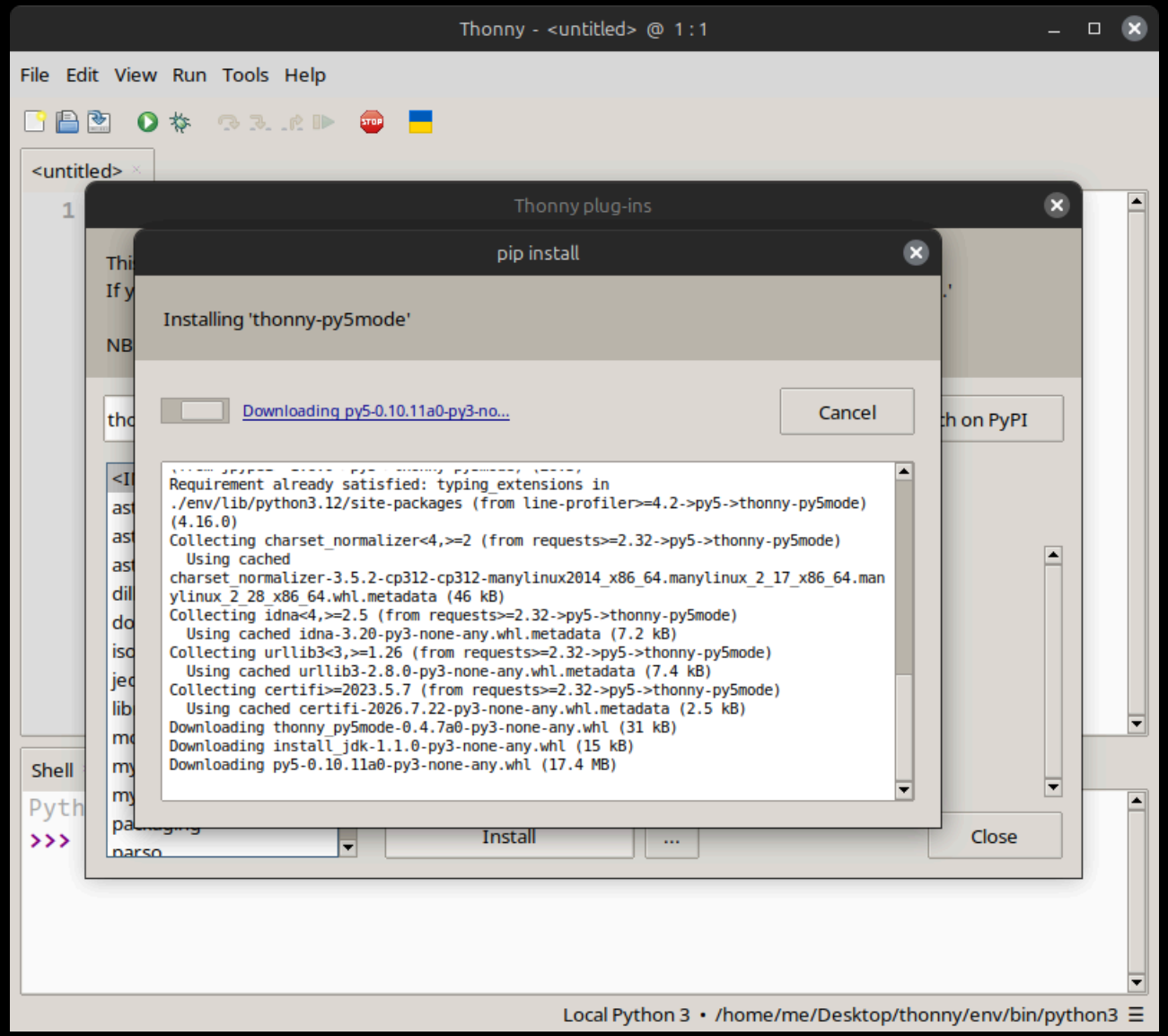
1. Download and run Thonny
2. Add plugin

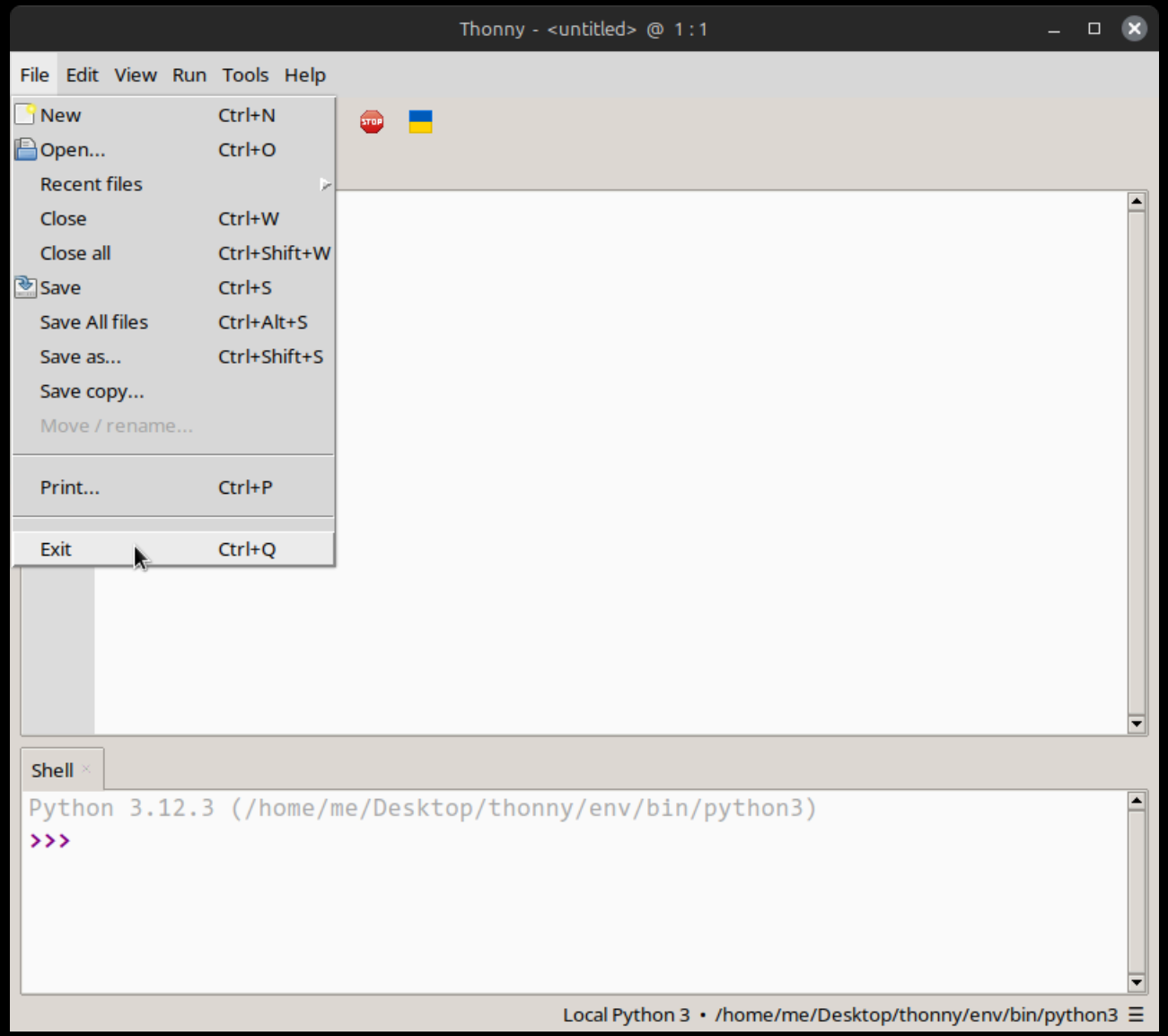

Language:
English (US) [default]
Initial settings:
Standard
Let's go!


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

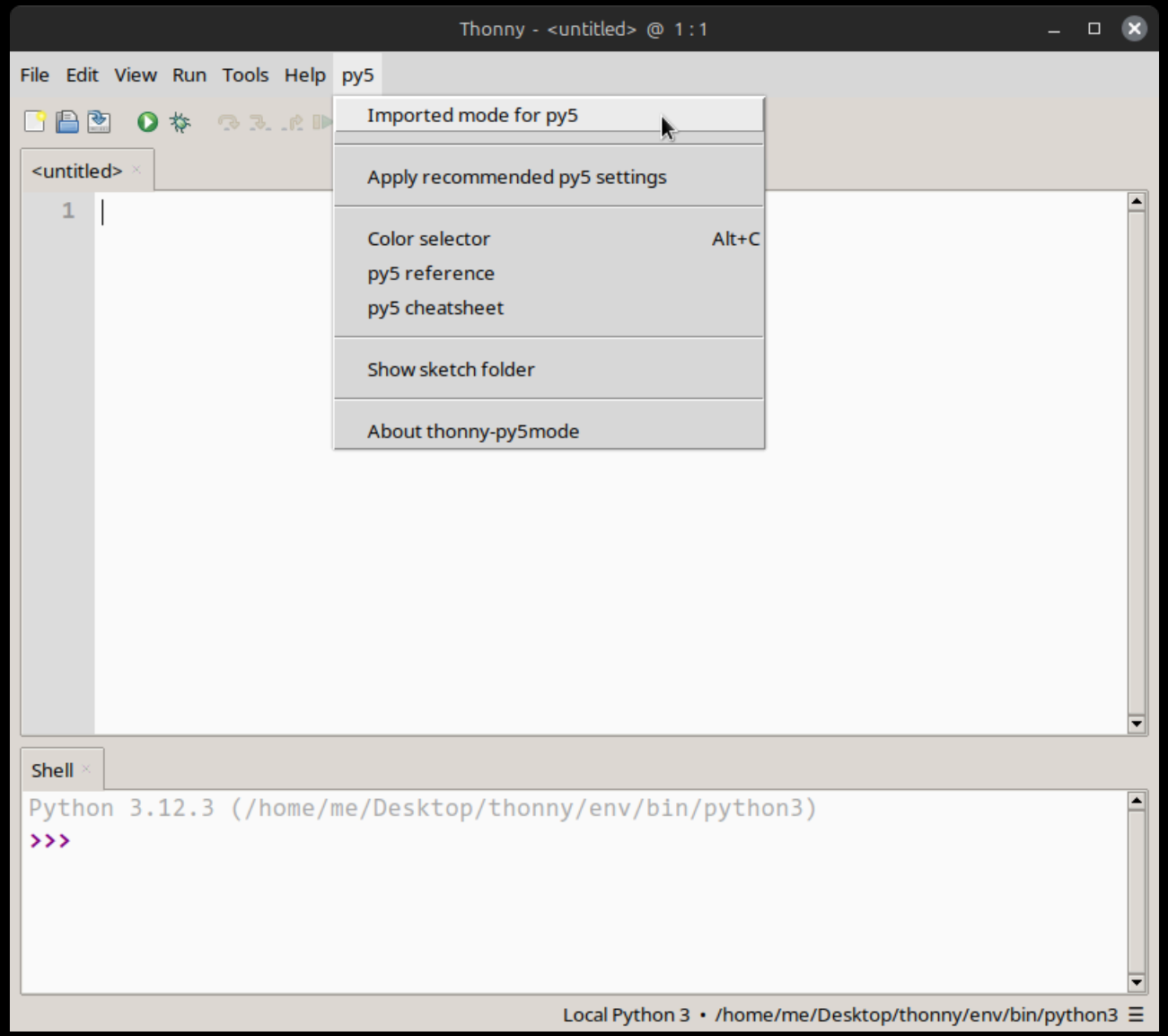

Thonny - <untitled> @ 1 : 1
File
Edit
View
Run
Tools
Help
Manage packages...
Open system shell...
Open Thonny program folder...
Open Thonny data folder...
Manage plug-ins...
Options...
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

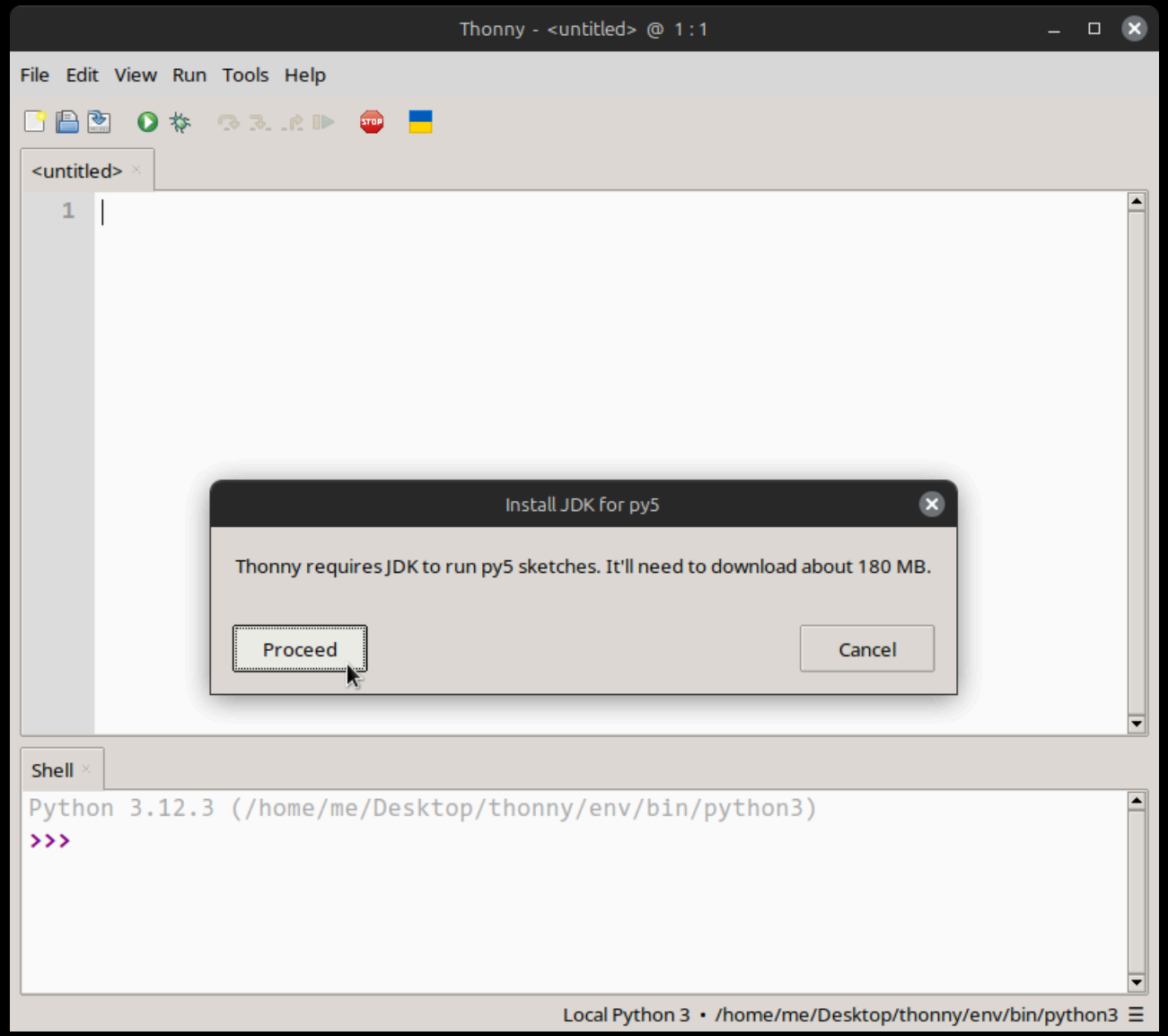

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
1
Thonny plug-ins
This dialog is for managing Thonny plug-ins and their dependencies.
If you want to install packages for your own programs then choose 'Tools → Manage packages...'
NB! You need to restart Thonny after installing / upgrading / uninstalling a plug-in.
thonny-py5mode
Search on PyPI
<INSTALL>
ast_serialize
astroid
asttokens
dill
docutils
isort
jedi
librt
mccabe
mypy
mypy_extensions
packaging
parso
Search results
thonny-py5mode
py5 mode plugin for Thonny
pyod
A Comprehensive and Scalable Python Library for Outlier Detection (Anomaly Detection)
python-monkey-business
Utility functions for monkey-patching python code
Close
Shell
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

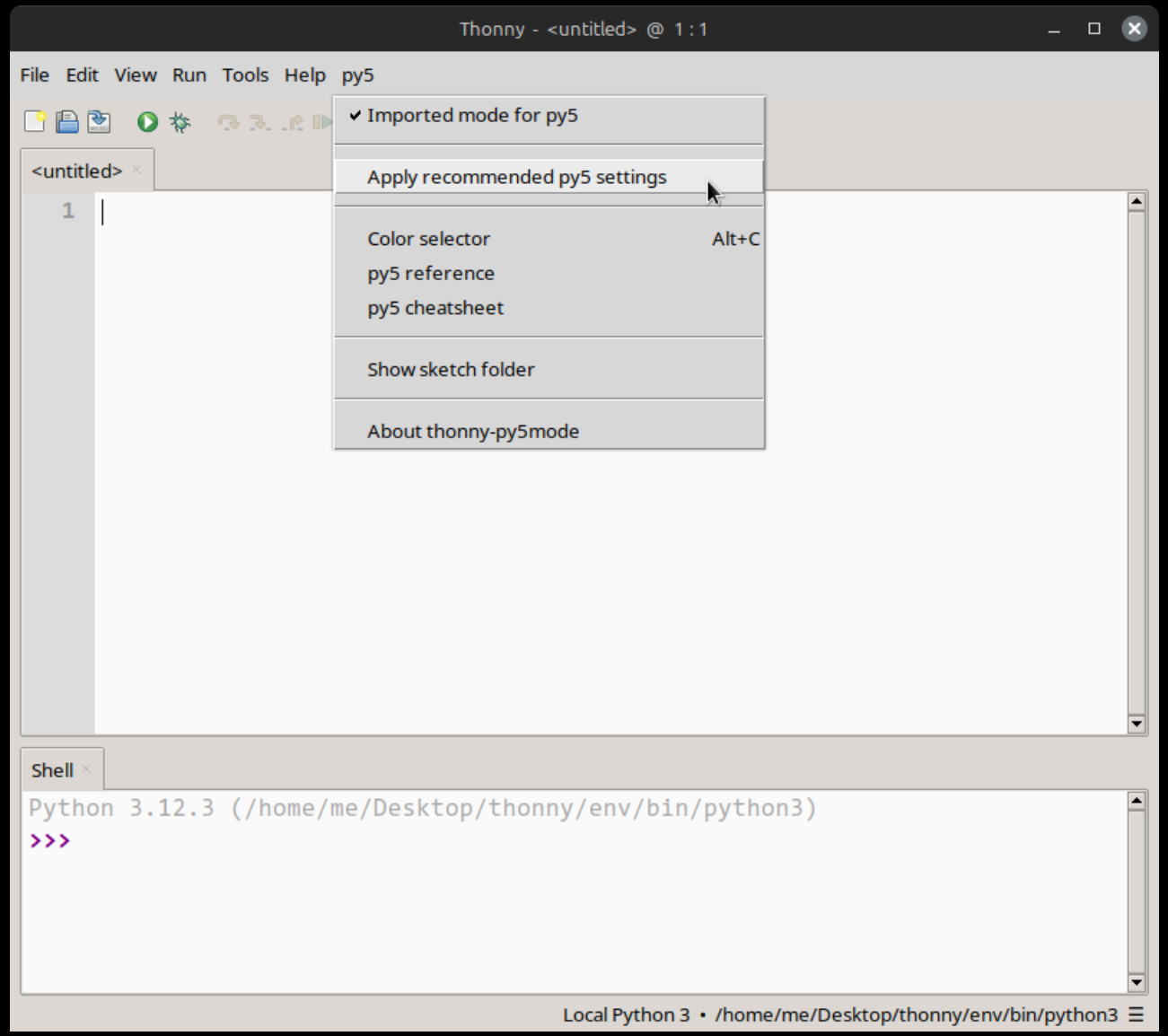

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
Thonny plug-ins
pip install
Installing 'thonny-py5mode'
Downloading py5-0.10.11a0-py3-no...
Cancel
Requirement already satisfied: typing_extensions in ./env/lib/python3.12/site-packages (from line-profiler>=4.2->py5->thonny-py5mode) (4.16.0)
Collecting charset_normalizer<4,>=2 (from requests>=2.32->py5->thonny-py5mode)
Using cached charset_normalizer-3.5.2-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (46 kB)
Collecting idna<4,>=2.5 (from requests>=2.32->py5->thonny-py5mode)
Using cached idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting urllib3<3,>=1.26 (from requests>=2.32->py5->thonny-py5mode)
Using cached urllib3-2.8.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi>=2023.5.7 (from requests>=2.32->py5->thonny-py5mode)
Using cached certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Downloading thonny_py5mode-0.4.7a0-py3-none-any.whl (31 kB)
Downloading install_jdk-1.1.0-py3-none-any.whl (15 kB)
Downloading py5-0.10.11a0-py3-none-any.whl (17.4 MB)
Install
...
Close
Shell
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

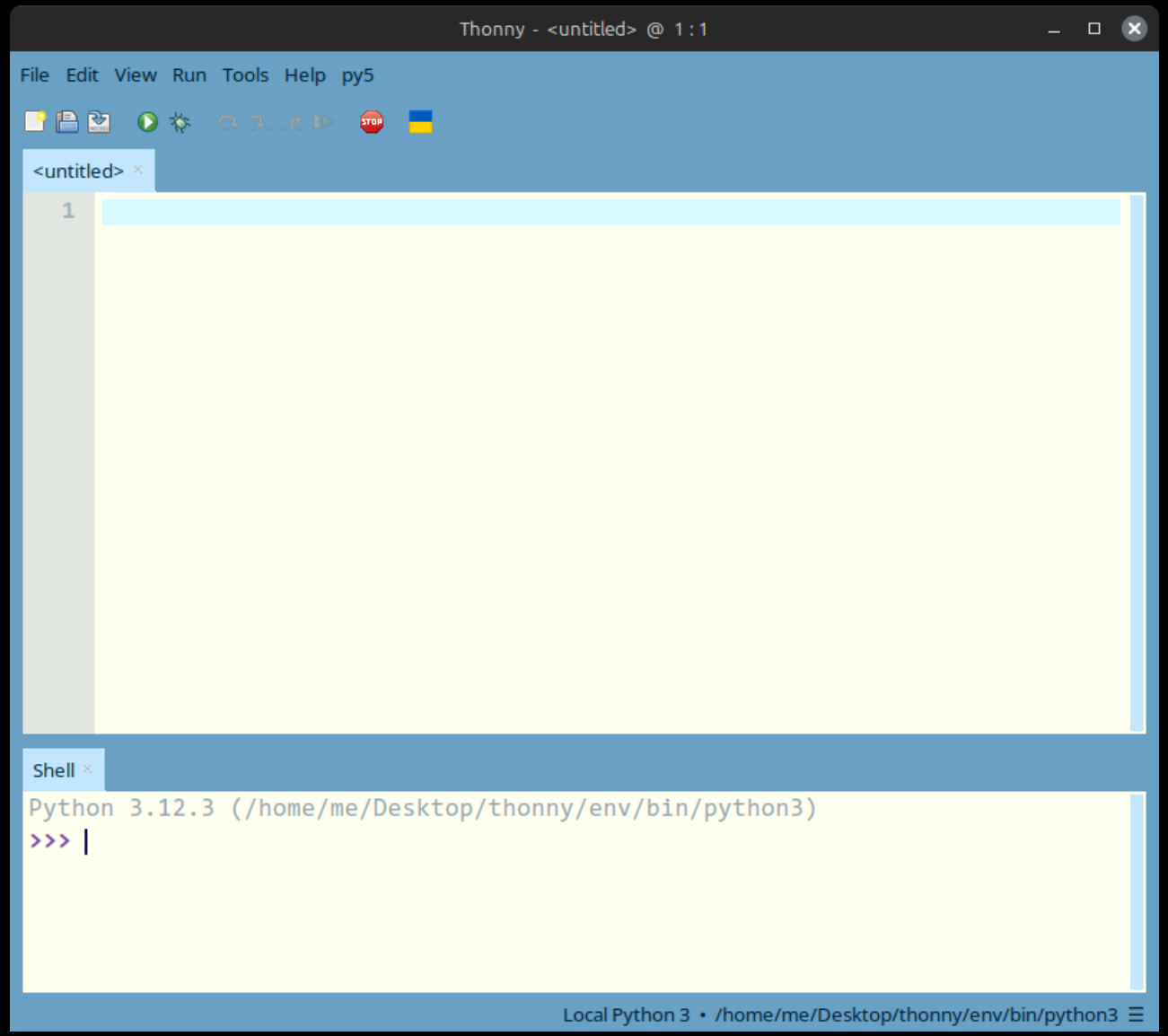

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
New Ctrl+N
Open... Ctrl+O
Recent files
Close Ctrl+W
Close all Ctrl+Shift+W
Save Ctrl+S
Save All files Ctrl+Alt+S
Save as... Ctrl+Shift+S
Save copy...
Move / rename...
Print... Ctrl+P
Exit Ctrl+Q
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

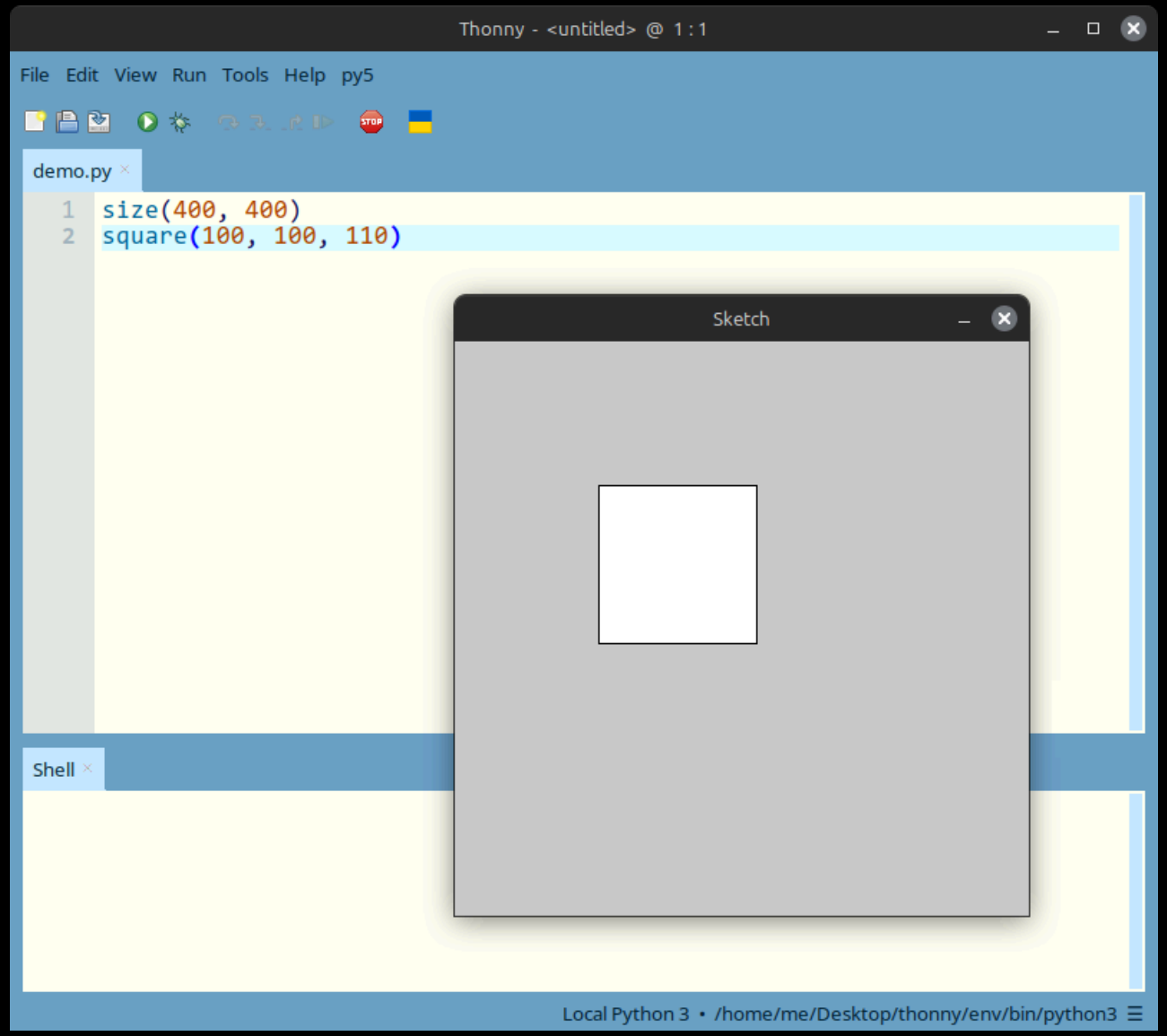
... restarting ...

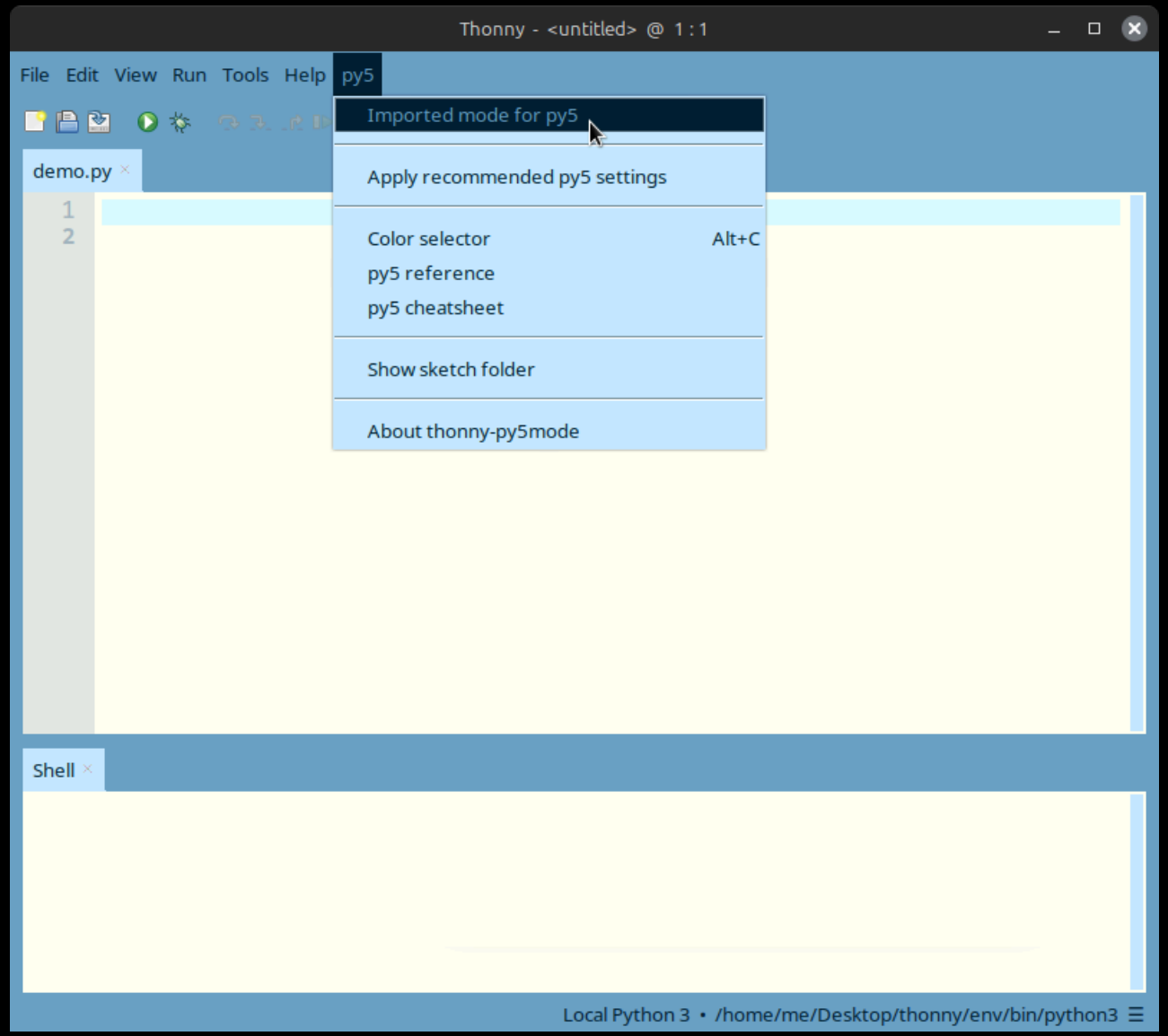

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
Imported mode for py5
Apply recommended py5 settings
Color selector Alt+C
py5 reference
py5 cheatsheet
Show sketch folder
About thonny-py5mode
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

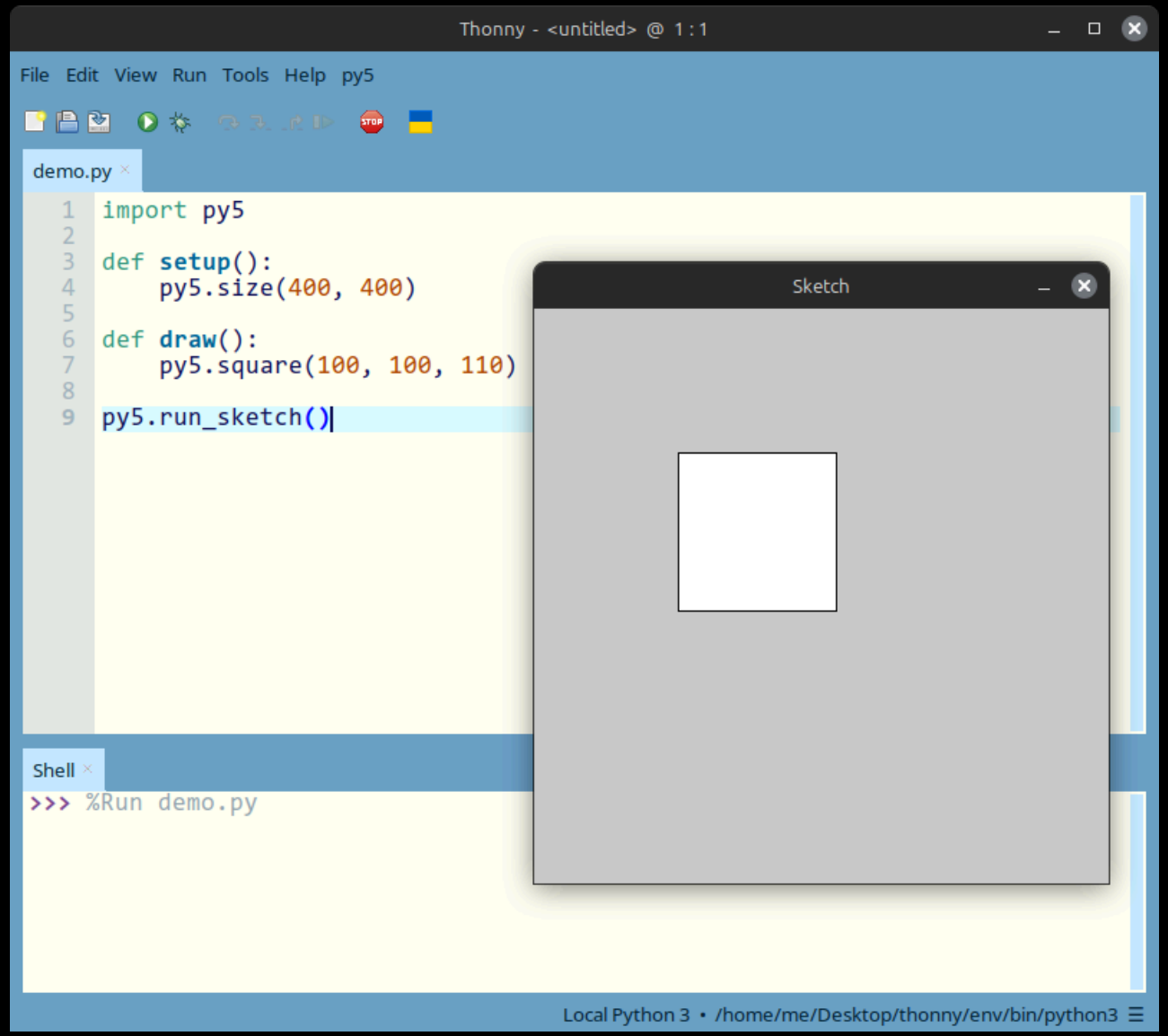

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
1
Install JDK for py5
Thonny requires JDK to run py5 sketches. It'll need to download about 180 MB.
Proceed
Cancel
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
Imported mode for py5
Apply recommended py5 settings
Color selector Alt+C
py5 reference
py5 cheatsheet
Show sketch folder
About thonny-py5mode
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
demo.py
1 size(400, 400)
2 square(100, 100, 110)
Sketch
Shell
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File
Edit
View
Run
Tools
Help
py5
Imported mode for py5
Apply recommended py5 settings
Color selector
Alt+C
py5 reference
py5 cheatsheet
Show sketch folder
About thonny-py5mode
demo.py
1
2
Shell
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
demo.py
1 import py5
2
3 def setup():
4 py5.size(400, 400)
5
6 def draw():
7 py5.square(100, 100, 110)
8
9 py5.run_sketch()
Sketch
Shell
>>> %Run demo.py
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

py5

py5 » Porting Approach

1

Translate Directly

Vectors, forces, particles: *near* line-for-line

py5 » Porting Approach

1

Translate Directly

Vectors, forces, particles: *near* line-for-line

2

Rethink Logic

Language differences and library substitutions

py5 » Porting Approach

1

Translate Directly

Vectors, forces, particles: *near* line-for-line

2

Rethink Logic

Language differences and library substitutions

3

Simplify Code

Python idioms to potentially simplify JS

1. Translate Directly

1. Translate Directly » py5 Naming

```javascript
// p5.js
function setup() {
  createCanvas(640, 240);
}
function draw() {
  circle(mouseX, mouseY, 48);
}
```

```python
# py5
def setup():
    size(640, 240)
def draw():
    circle(mouse_x, mouse_y, 48)
```

1. Translate Directly » Key Renames to Know

p5.js	py5	Why
`createCanvas()`	`size()`	Processing Java
`mouseX`, `strokeWeight()`	`mouse_x`, `stroke_weight()`	snake_case
`map()`	`remap()`	Python built-in
NOC `next`, `len` variables	`next_`, `len_`	Python built-in
`createVector`	`Py5Vector2D`, `Py5Vector3D`	real class
`random()`	also `random()`	py5 built-in
More on slides to come ...		

1. Translate Directly » Randomness & Noise

```javascript
// p5.js
let x = random(width);
let g = randomGaussian(0, 1);
let n = noise(xoff);
let coinflip = floor(random(2));  // 0 or 1
```

```python
# py5
x = random(width)                  # py5's own, not the random module
g = random_gaussian(0, 1)
n = noise(xoff)
coinflip = random_int(1)           # inclusive: returns 0 or 1
```

1. Translate Directly » Vector Operations

```p5
// p5.js
position = createVector(100, 100);
velocity = createVector(2.5, 2);
position.add(velocity);
print(position.mag());
position.setMag(10);
```

```py5
# py5
position = Py5Vector2D(100, 100)
velocity = Py5Vector2D(2.5, 2)
position += velocity
print(position.mag)
position.mag = 10
```

1. Translate Directly » Multi-File Caveat

sketch.py

```
from mover import Mover
...

def setup():
    ...
    mover = Mover()
    ...

def draw():
    ...
    mover.show()
```

1. Translate Directly » Multi-File Solution #1

mover.py

```
from py5 import *

class Mover:
    def __init__(self, width, height):  # must pass in width & height
        self.width = width
        self.height = height
        self.position = Py5Vector2D(self.width / 2, self.height / 2)

    def show(self):
        fill(127)
        circle(self.position.x, self.position.y, 48)
```

1. Translate Directly » Multi-File Solution #2

mover.py

```
from py5 import *

class Mover:
    def __init__(self):
        s = get_current_sketch()
        self.position = Py5Vector2D(s.width / 2, s.height / 2)

    def show(self):
        fill(127)
        circle(self.position.x, self.position.y, 48)
```

1. Translate Directly » Multi-File Solution #3

mover.py

```
# PY5 IMPORTED MODE CODE

class Mover:
    def __init__(self):
        self.position = Py5Vector2D(width / 2, height / 2)

    def show(self):
        fill(127)
        circle(self.position.x, self.position.y, 48)
```

NameError: The name "Py5Vector2D" is not defined. Your Sketch is also running in Imported Mode. If the code throwing this exception was imported into your main py5 Sketch code, please ensure the py5 Imported Mode marker "# PY5 IMPORTED MODE CODE" has been properly added to the module.

2. Rethink Logic

2. Rethink Logic » Scope Is Different

```
// p5.js -- outer variables just work
let position;

function draw() {
    position.add(velocity);
}
```

```
# py5 -- assignment needs an explicit declaration
def draw():
    global position
    position += velocity
```

2. Rethink Logic » No preload()

```p5
// p5.js -- assets load asynchronously before setup()
let img;

function preload() {
  img = loadImage('texture.png');
}
```

```py5
# py5 -- loading is synchronous: just do it in setup()
def setup():
    global img
    size(640, 240)
    img = load_image('texture.png')
```

2. Rethink Logic » Matter.js to Pymunk

```javascript
// p5.js
const { Engine, Bodies, Composite } = Matter;

let engine = Engine.create();

let box = Bodies.rectangle(100, 100, 50, 50, { friction: 0.01 });
Composite.add(engine.world, box);
```

```python
# py5 -- pymunk separates Body (physics) from Shape (collision skin)
from pymunk import Body, moment_for_box, Poly, Space

engine = Space()
engine.gravity = (0, 1.0 * SCALE_GRAVITY)

box_body = Body(mass, moment_for_box(mass, (w, h)))
box_body.position = (100, 100)
box_shape = Poly.create_box(box_body, (w, h))
box_shape.friction = 0.01 * SCALE_FRICTION
engine.add(box_body, box_shape)
```

2. Rethink Logic » Matter.js Calibration

pymunk_constants.py

```
"""
Since the Matter.js and Pymunk engines use different units and solvers,
these values provide an *approximate* conversion for matching visual behavior.
"""

SCALE_GRAVITY = 900.0
FPS = 60.0
DT = 1.0 / FPS
SCALE_TIME = FPS
SCALE_VELOCITY = SCALE_TIME
SCALE_ANG_VELOCITY = SCALE_TIME
SCALE_FRICTION = SCALE_TIME
```

2. Rethink Logic » Toxiclibs (jars)

Simply skip the JS Toxiclibs port and use the Java original.

```
"""
NOTE:
Examples 6.11 through 6.15 (end of chapter) use the Toxiclibs Java libraries.
For details on py5's integration with Processing (Java) libraries, see:
https://py5coding.org/how_tos/use_processing_libraries.html
"""

from toxi.geom import Vec2D, Rect
from toxi.physics2d import VerletPhysics2D, VerletParticle2D, VerletSpring2D
from toxi.physics2d.behaviors import GravityBehavior2D
```

2. Rethink Logic » Toxiclibs Composition

```javascript
// p5.js -- subclass the toxiclibs.js particle directly
class Particle extends VerletParticle2D {
  constructor(x, y, r) {
    super(x, y); this.r = r;
  }
  show() {
    circle(this.x, this.y, this.r * 2);
  }
}
physics.addParticle(particle);  // pass the object itself
```

```python
# py5 -- wrap the Java particle via composition
class Particle:
    def __init__(self, x, y, r):
        self.p = VerletParticle2D(x, y)  # underlying Java object
        self.r = r
    def show(self):
        circle(self.p.x(), self.p.y(), self.r * 2)

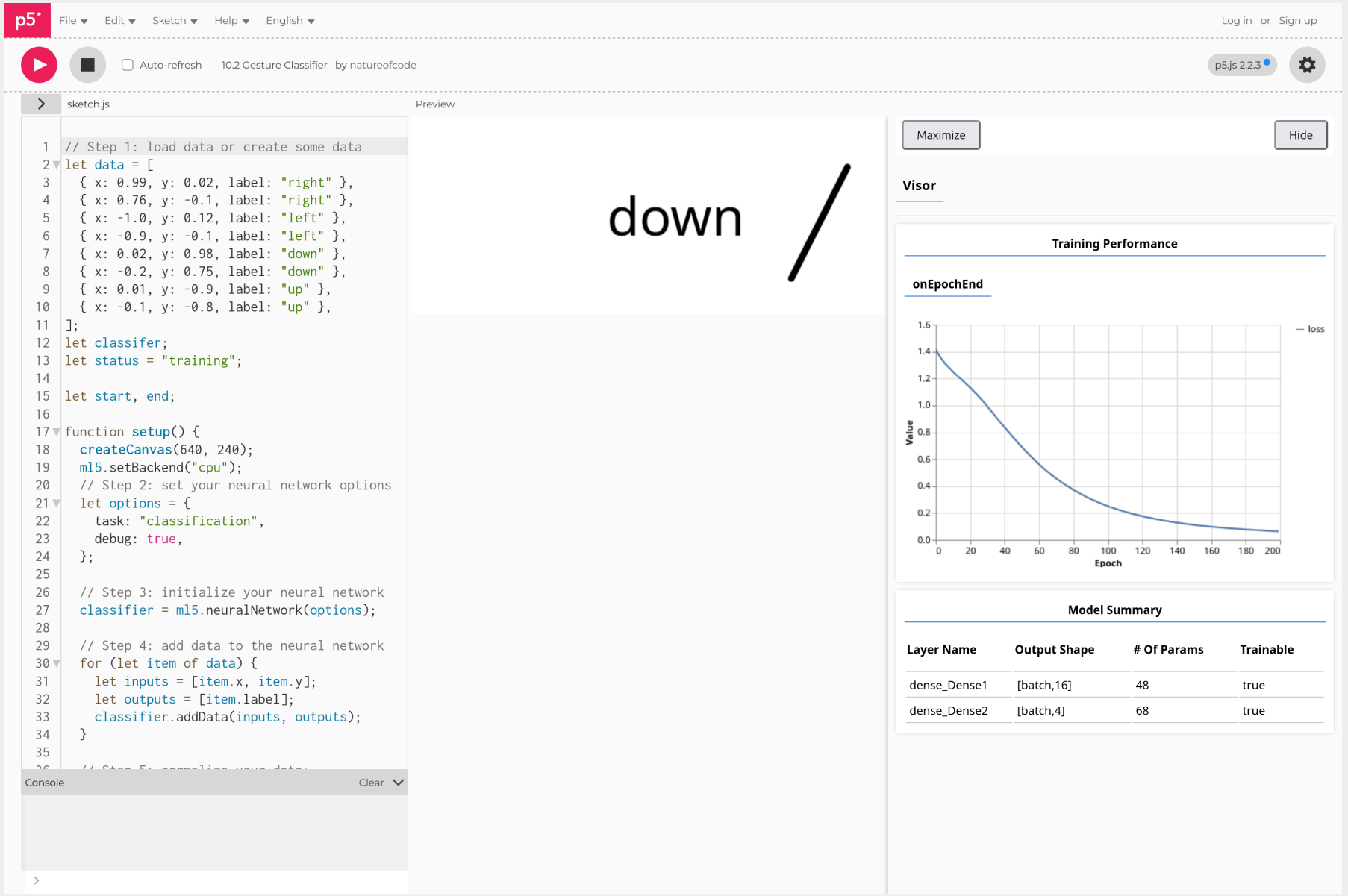
physics.addParticle(particle.p)  # pass the Java object via .p
```

2. Rethink Logic » ml5.js to scikit-learn

```
// p5.js -- async training with callbacks
classifier = ml5.neuralNetwork({ task: "classification" });
classifier.addData(inputs, outputs);
classifier.train({ epochs: 200 }, finishedTraining);
// ...must wait for finishedTraining() callback
```

```
# py5 -- synchronous: trains before the window opens
from sklearn.neural_network import MLPClassifier

classifier = MLPClassifier(hidden_layer_sizes=(16,), max_iter=2500)
classifier.fit(inputs, outputs)  # blocks until done
```




```
// Step 1: load data or create some data
let data = [
  { x: 0.99, y: 0.02, label: "right" },
  { x: 0.76, y: -0.1, label: "right" },
  { x: -1.0, y: 0.12, label: "left" },
  { x: -0.9, y: -0.1, label: "left" },
  { x: 0.02, y: 0.98, label: "down" },
  { x: -0.2, y: 0.75, label: "down" },
  { x: 0.01, y: -0.9, label: "up" },
  { x: -0.1, y: -0.8, label: "up" },
];
let classifer;
let status = "training";

let start, end;

function setup() {
  createCanvas(640, 240);
  ml5.setBackend("cpu");
  // Step 2: set your neural network options
  let options = {
    task: "classification",
    debug: true,
  };

  // Step 3: initialize your neural network
  classifier = ml5.neuralNetwork(options);

  // Step 4: add data to the neural network
  for (let item of data) {
    let inputs = [item.x, item.y];
    let outputs = [item.label];
    classifier.addData(inputs, outputs);
  }
```

10.02 (gesture_classifier.py)



10.02 (gesture_classifier.py)

2. Rethink Logic » Neuroevolution (NumPy Brain)

brain_ne.py

```python
# ml5.js-style neural network functionality implemented with bespoke class.

import numpy as np

class Brain:
    """Vectorized NumPy brain with GA helpers."""

    def __init__(self, inputs: int, outputs: int, hidden: int = 8) -> None:
        ...

    def forward(self, inputs: list[float]) -> list[float]:
        ...

    def classify_binary(self, inputs: list[float], threshold: float = 0.5) -> bool:
        ...

    def predict_continuous_01(self, inputs: list[float]) -> list[float]:
        ...
```

3. Simplify Code

3. Simplify Code » Operator Overloading

```
// p5.js -- verbose static/method calls
let force = p5.Vector.sub(attractor, mover);
force.normalize();
force.mult(strength);
mover.acceleration.add(force);
```

```
# py5 -- reads more like the maths
force = (attractor - mover).norm * strength
mover.acceleration += force
```

3. Simplify Code » Comprehensions

```javascript
// p5.js
let training = [];
for (let i = 0; i < 2000; i++) {
  training.push([random(-width/2, width/2), random(-height/2, height/2), 1]);
}
```

```python
# py5
training = [
  [random(-width / 2, width / 2), random(-height / 2, height / 2), 1]
  for _ in range(2000)
]
```

3. Simplify Code » Backward Filter

```p5
// p5.js -- loop backward to splice safely
for (let i = particles.length - 1; i >= 0; i--) {
  particles[i].run();
  if (particles[i].isDead()) particles.splice(i, 1);
}
```

```py5
# py5 -- same backward-loop technique works ...
for i in range(len(particles) - 1, -1, -1):
    particles[i].run()
    if particles[i].dead: del particles[i]
```

If you're wondering about ***.dead*** *...*

3. Simplify Code » Properties over Methods

```
// p5.js -- isDead() is a method call
if (particles[i].isDead()) particles.splice(i, 1);
...
isDead() {
  return (this.lifespan < 0.0);
}
```

```
# py5 -- .dead is a @property (no parentheses)
if particles[i].dead: del particles[i]
...
@property
def dead(self) -> bool:
    return self.lifespan < 0.0
```

3. Simplify Code » Slice Notation

```p5
// p5.js
cells = new Array(floor(width / w)).fill(0);
let nextgen = cells.slice();

let [left, me, right] = cells.slice(i - 1, i + 2);
```

```py5
# py5
cells = [0] * (width // w)
next_gen = cells[:]

left, me, right = cells[i - 1 : i + 2]
```

3. Simplify Code » Small Quality-of-Life Wins

Python / py5	Why
`position += velocity`	Augmented assignment on vectors
`Particle(*self.origin)`	Vectors unpack as iterables
`10_000`	Readable numeric literals
`def step(self) -> None:`	Type hints document intent
No `let` / `const` / `;` / `{}`	Less syntactic noise
Synchronous ML training	No callback pyramids

Materials

Dedication
Acknowledgments
Introduction
0 Randomness
1 Vectors
2 Forces
3 Oscillation
4 Particle Systems
5 Autonomous Agents
6 Physics Libraries
7 Cellular Automata
8 Fractals
9 Evolutionary Computing
10 Neural Networks
11 Neuroevolution
Appendix: Creature Design
Additional Resources

Additional Resources

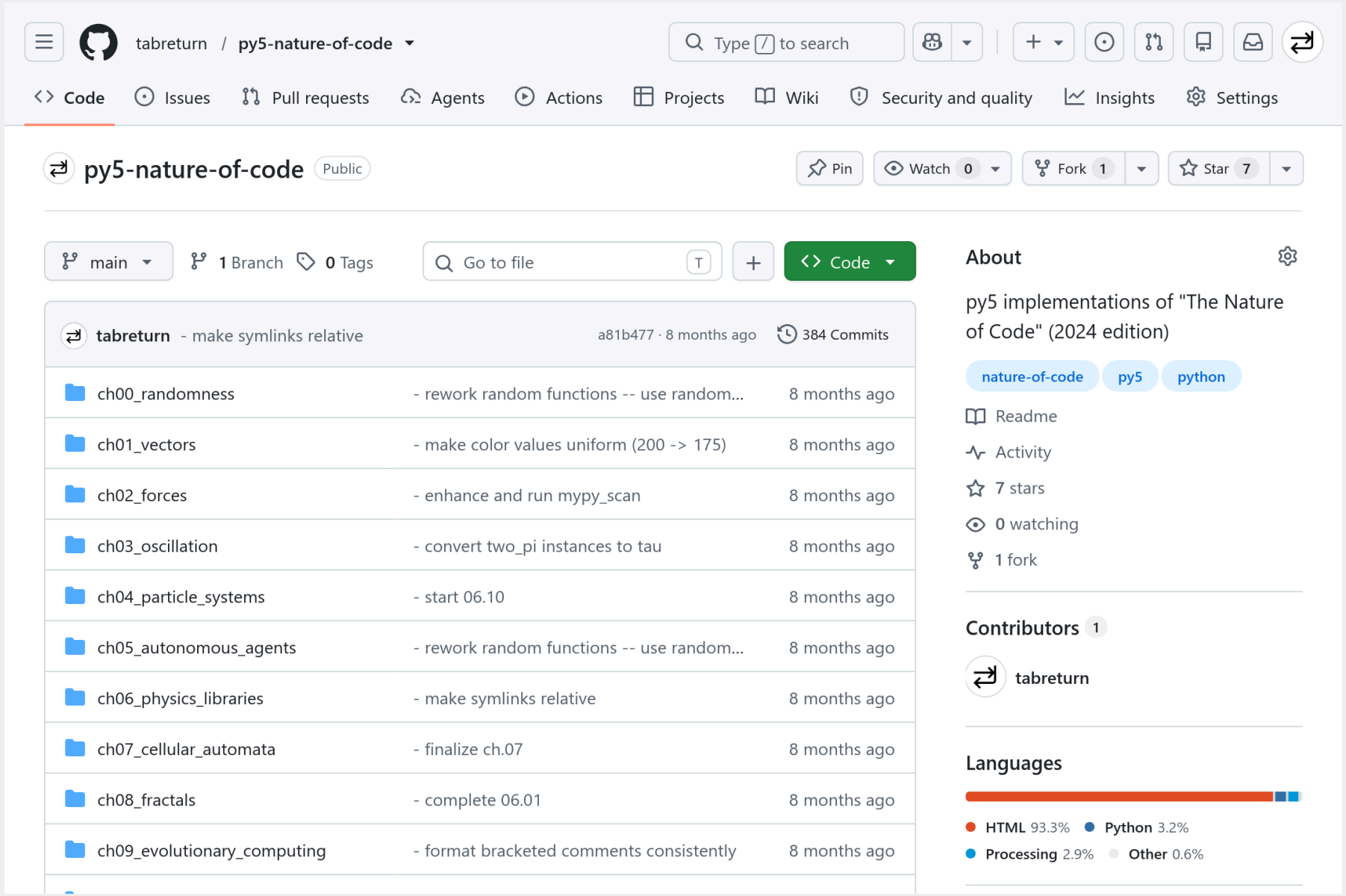
Ports of Code Examples to Other Languages

- Processing (Java) port by Coding Train community
- Python port with pygame by Niklas Hohn
- Another Python port with pygame by Oenarion
- py5 port (a version of Processing for Python) by tabreturn
- C++ port with openFrameworks by Jack B. Du
- C++/SDL by Alcides Schulz
- Go + Ebitengine by Jordan Philyaw
- If you are working on a port, please open an issue on GitHub with links and other relevant information!

Further Reading

Books

- Alexander, R. McNeill. *Principles of Animal Locomotion*. Princeton, NJ: Princeton University Press,



https://github.com/tabreturn/py5-nature-of-code

banner.png	- wip (11.02)	9 months ago
mypy_scan.py	- amend mypy_scan to handle getters/set...	8 months ago

README

Nature of Code (2), py5 Port

py5 implementations of Daniel Shiffman's "The Nature of Code" (2024 edition)

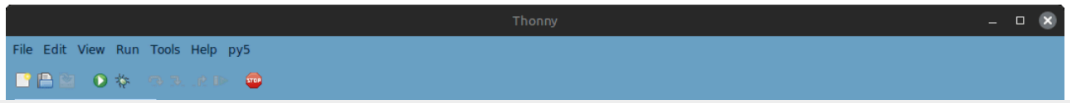
Built with py5 using Thonny-py5mode

I've taken a few liberties in my interpretation. First, the examples in the book and its accompanying p5.js sketches don't always align exactly. Second, in places, minor adjustments were necessary to support an idiomatic, py5-compatible Python implementation. Some chapters rely on p5.js-exclusive libraries, so I've had to employ substitutes/alternatives. Throughout, however, my goal has been to balance faithfulness to the original material with Python best practices and clarity for readers working through the book.

To run the sketches, install Thonny-py5mode and activate **py5 → Imported mode for py5** in the editor.

Find the original p5.js examples in the noc-book-2 repo, linked alongside each py5 sketch listed below, or in the Book's online Examples section.

Where text is displayed, the sketches use the *DejaVu Sans Mono* font.



Jekyll using Docker image
Package a Jekyll site using the jekyll/builder Docker image.
By GitHub Actions
Configure

Clojure
Build and test a Clojure project with Leiningen.
By GitHub Actions
Configure

More workflows

https://github.com/tabreturn/py5-nature-of-code

README

You'll see this marker in several modules, so that there's no need to add `from py5 import *` or `import py5`.

⚙️ Part 1: Inanimate Objects

- ▸ Ch. 00 - Randomness
- ▸ Ch. 01 - Vectors
- ▸ Ch. 02 - Forces
- ▸ Ch. 03 - Oscillation
- ▸ Ch. 04 - Particle Systems

🧬 Part 2: It's Alive!

- ▸ Ch. 05 - Autonomous Agents
- ▸ Ch. 06 - Physics Libraries
- ▸ Ch. 07 - Cellular Automata
- ▸ Ch. 08 - Fractals

🧠 Part 3: Intelligence

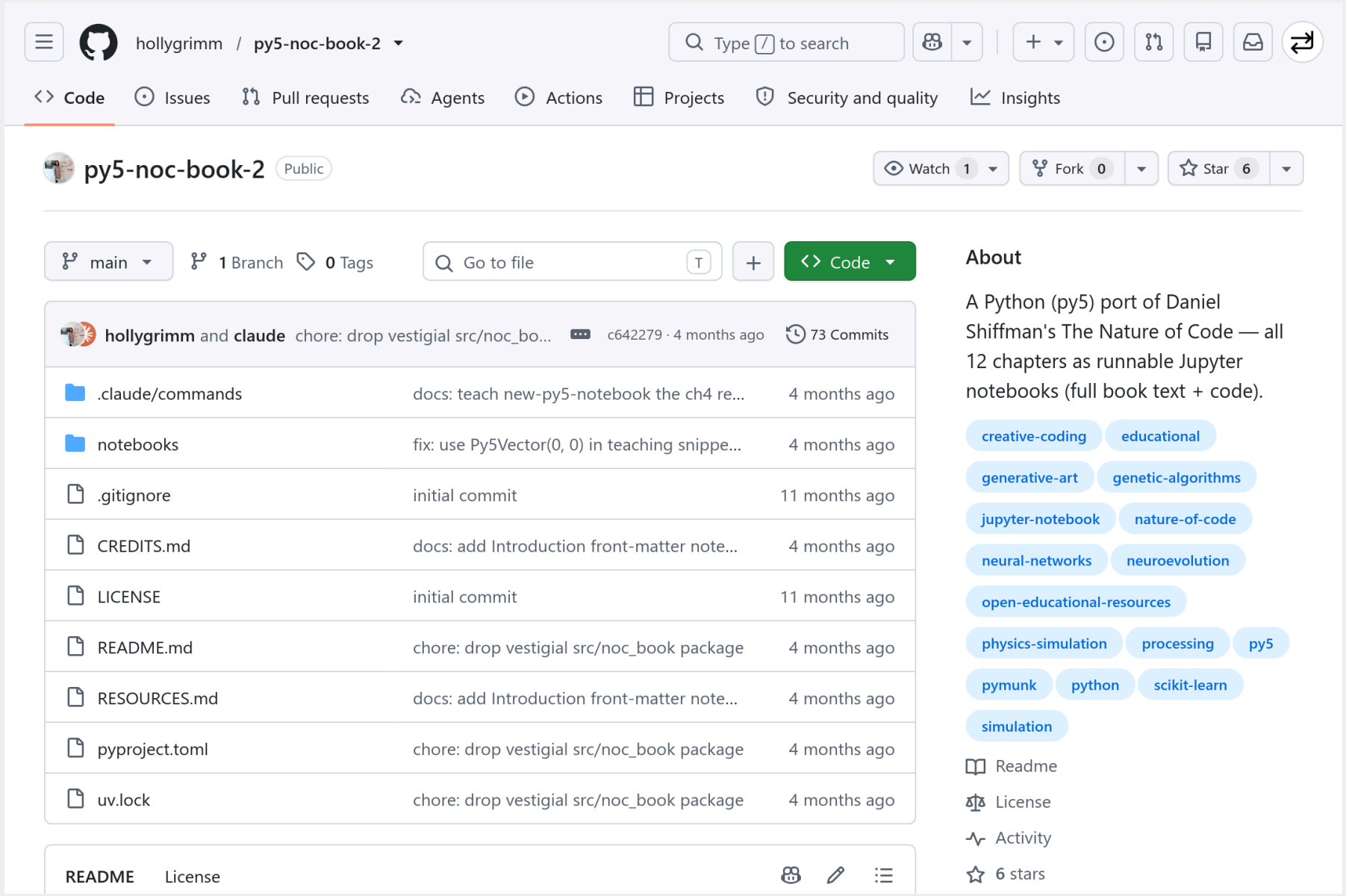
https://github.com/tabreturn/py5-nature-of-code

Materials » Jupyter py5 Port

by @hollygrimm

https://github.com/hollygrimm/py5-noc-book-2

Jupyter notebooks (`.ipynb`) · Module mode (`import py5`) · Pymunk for all of ch06; no Toxiclibs · similar scikit-learn + hand-rolled NumPy approach.



https://github.com/hollygrimm/py5-noc-book-2

main py5-noc-book-2 / notebooks / 00_randomness.ipynb Top

Preview Code Blame 1916 lines (1916 loc) · 85.8 KB Raw

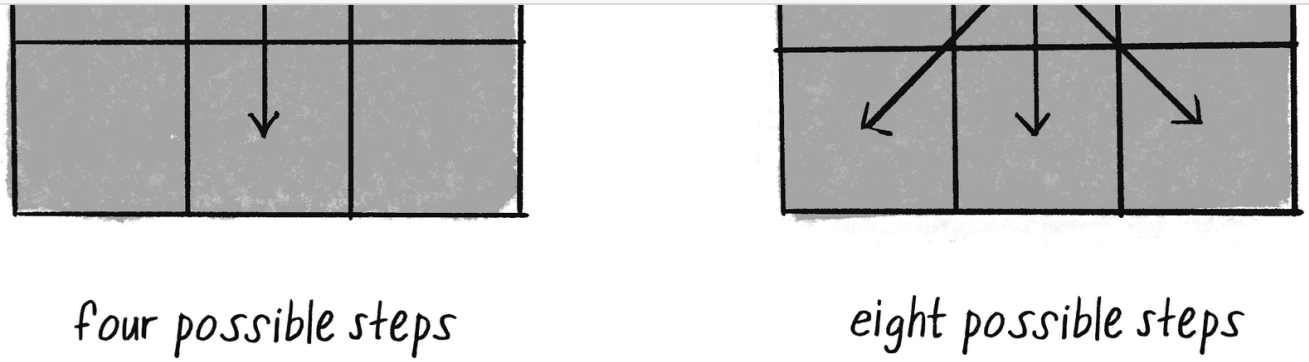


Figure 0.1: The steps of a random walker, with and without diagonals

To implement a `Walker` object that can step to any neighboring pixel (or stay put), I could pick a number from 0 to 8 (nine possible choices). However, another way to write the code would be to pick from three possible steps along the x-axis (–1, 0, or 1) and three possible steps along the y-axis:

In [ ]:

```
def step(self):
    # Yields –1, 0, or 1
    xstep = int(py5.random(3)) - 1
    ystep = int(py5.random(3)) - 1
    self.x += xstep
    self.y += ystep
```

Taking this further, I could get rid of `int()` and use the `random()` function's original floating-point numbers to create a continuous range of possible step lengths from –1 to 1, as shown next.

In [ ]:

```
def step(self):
    # Any floating-point number from –1 to 1
    xstep = py5.random(-1, 1)
```

https://github.com/hollygrimm/py5-noc-book-2

End